KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions"

Submitted as a part of the course

AGENTIC AI

Submitted By

Student Name	Enrollment No.
_____ _	_____

Program: B.Tech CSE / B.Tech CSE-AIML
Semester: VI
Section: _____

Under the Guidance of

Faculty Name
Assistant Professor
Department of Computer Science & Information Technology

Karnavati University, Gandhinagar

Academic Year: 2026–27

CERTIFICATE

This is to certify that the mini project entitled **“AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions”** has been successfully completed by the following students of B.Tech AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions Semester VI, Karnavati University, as a part of the course **Agentic AI** during the academic year 2026–27.

Sr. No.	Student Name	Enrollment No.
1		
2		
3		
4		

The project has been carried out under the guidance and supervision of the faculty member. The work presented in this report is the original work carried out by the students as part of their mini project.

Faculty
Name:
Signature:
Date: _____

Guide
_____
_____

DECLARATION

We hereby declare that the mini project entitled **"AeroMind: Multi-Agent Post-Flight Analysis of ArduPilot Drone Missions"** submitted as part of the course **Agentic AI** is our original work carried out under the guidance of the faculty member.

We further declare that this work has not been submitted previously to any other institution or university for the award of any degree, diploma, or academic certification.

All research papers, datasets, frameworks, APIs, libraries, websites, and other resources used during the development of the project have been appropriately acknowledged and referenced.

Sr. No.	Student Name	Enrollment No.
1		
2		
3		
4		

Date: _____

Place: Gandhinagar

ACKNOWLEDGEMENT

We thank our faculty guide for the direction, feedback and patience given throughout this mini project, and the Department of Computer Science & Information Technology, Unitedworld Institute of Technology, Karnavati University, for including Agentic AI in the curriculum and for the laboratory and computing facilities that made the work possible.

The simulation work builds on the disaster-zone world and the Hexa-X airframe our team developed for the Smart India Hackathon 2026 (the Aero Sense project), and on open-source software from the ArduPilot, Gazebo and Python communities: ArduPilot SITL, MAVProxy and pymavlink, Gazebo Harmonic, Streamlit, Plotly, pandas, Pydantic and Ollama. We acknowledge the authors of the third-party simulation models listed in the references.

Finally, we thank every team member for their contribution to the design, implementation, testing and documentation of AeroMind.

ABSTRACT

After a drone flight, the information needed to understand what happened is scattered across a telemetry log, a mission file, a parameter file and the pilot's memory, and reading it takes expertise. Most operators therefore never learn why a flight drifted, ran its battery low or lost its telemetry link, and repeat the same mistakes. This project presents **AeroMind**, a multi-agent Agentic AI system that analyses the files of one ArduPilot flight and produces a one-page verdict, a phase-by-phase story, a telemetry-link timeline, root causes, a next-mission risk score and a corrected mission file.

Deterministic Python tools convert MAVLink logs into verified metrics with Good/Caution/Bad statuses. An orchestrator builds a manifest, obtains a plan from a large language model (LLM), checks the plan against rules, and runs five specialist agents (Flight Summary, Telemetry Link, Diagnosis, Risk and Advisor) on a shared state. A deterministic Verifier rejects any claim not supported by the data and triggers one self-correction round. A flight history provides long-term memory: personal baselines, battery prediction and recurring issues. The system is built in Python with Pydantic, Streamlit and Plotly, and runs with Claude through the Anthropic API or a local Qwen2.5-3B model through Ollama.

Eight flights were flown in ArduPilot SITL in a Gazebo disaster world with injected faults and known ground truth. AeroMind's agents detected all four injected faults with the correct root cause, located both telemetry dropouts within 0.1 s of the truth, raised no false alarm on the baseline flight and ranked next-mission risk correctly. The mission they corrected for the undersized-battery scenario raised the battery at landing from 14 % to 25 %, and the speed fix reduced the wind-scenario cross-track error from 1.8 m to 1.44 m. The work shows that LLM agents paired with deterministic verification produce useful, grounded flight analysis even with a small model running locally.

TABLE OF CONTENTS

ACKNOWLEDGEMENT....4
ABSTRACT....5
LIST OF FIGURES....8
LIST OF TABLES....9
CHAPTER 1 – INTRODUCTION....10
1.1 Introduction....10
1.2 Background....10
1.3 Problem Statement....11
1.4 Motivation....11
1.5 Objectives....12
1.6 Scope of the Project....12
CHAPTER 2 – LITERATURE REVIEW....13
2.1 Existing Systems and Research....13
2.2 Comparative Analysis....13
2.3 Research Gap....14
CHAPTER 3 – PROPOSED SYSTEM....15
3.1 Proposed Solution....15
3.2 System Requirements....15
3.3 Functional Requirements....16
3.4 Non-Functional Requirements....17
CHAPTER 4 – AGENT ARCHITECTURE....18
4.1 Overall System Architecture....18
4.2 Architecture Components....18
4.3 Agent Roles and Responsibilities....19
4.4 Agent Workflow....19
CHAPTER 5 – AGENTIC AI CONCEPTS USED....22
5.1 ReAct....22
5.2 Planning....22
5.3 Tool Calling....23
5.4 Memory....24
5.5 Retrieval-Augmented Generation....24
5.6 Multi-Agent System....24
5.7 Human-in-the-Loop....25
CHAPTER 6 – METHODOLOGY....26
6.1 Input Processing....26
6.2 Task Understanding....26
6.3 Task Planning....27
6.4 Reasoning and Decision Making....27

6.5 Tool Selection....27
6.6 Task Execution....27
6.7 Observation and Feedback....28
6.8 Final Response Generation....29
6.9 Evaluation Data: Simulated Flights with Known Faults....29
CHAPTER 7 – IMPLEMENTATION....31
7.1 Project Structure....31
7.2 Agent Implementation....31
7.3 Tool Implementation....32
7.4 Memory Implementation....33
7.5 RAG Implementation....33
7.6 User Interface....33
CHAPTER 8 – RESULTS AND OUTPUT....36
8.1 Test Cases....36
8.2 Sample Outputs....36
8.3 Agent Execution Trace....37
CHAPTER 9 – PERFORMANCE EVALUATION....39
CHAPTER 10 – CHALLENGES AND LIMITATIONS....42
CHAPTER 11 – FUTURE SCOPE....43
CHAPTER 12 – CONCLUSION....44
REFERENCES....45
APPENDIX....47
Appendix A – Source Code....47
Appendix B – Additional Screenshots....47
Appendix C – Sample Prompts....52
Appendix D – Test Data....53

LIST OF FIGURES

Figure 1: Overall system architecture of AeroMind.....18
Figure 2: Agent workflow of one analysis.....21
Figure 3: Planning and execution flow of the orchestrator.....23
Figure 4: Simulation and data-collection pipeline.....29
Figure 5: Demo route in the disaster world (structures from the world file).....30
Figure 6: Home page: input form (sidebar) and the Summary tab for S4.....34
Figure 7: Agents tab (S3): orchestrator plan, Verifier verdicts and each agent's execution.....35
Figure 8: Ask tab (S3): an answer from the local LLM, verified against the cited metrics.....35
Figure 9: Generated report.html for S2.....37
Figure 10: Track tab (S3): planned route, flown path coloured by link quality, dropout pins. 48
Figure 11: Telemetry tab (S3): link strip with both dropouts, message rate, dropout table.....49
Figure 12: History tab (S2 fixed): baseline, trends and what worked before.....50
Figure 13: Power & attitude tab (S2): battery falling to 14 %.....51
Figure 14: Summary tab (S3): both dropouts on the telemetry strip.....52

LIST OF TABLES

Table 1: Literature review summary.....13
Table 2: Comparison of existing tools and the proposed system.....13
Table 3: Hardware requirements.....16
Table 4: Software requirements and technologies used.....16
Table 5: Functional requirements.....16
Table 6: Architecture components.....18
Table 7: Agent roles and responsibilities.....19
Table 8: Tools: purpose, input and output.....23
Table 9: Metrics and thresholds.....26
Table 10: Base risk rules (Low < 30, Medium 30–59, High ≥ 60, capped at 100).....27
Table 11: Mission edit actions and limits.....28
Table 12: Evaluation scenarios.....30
Table 13: Test cases and actual results.....36
Table 14: Success metrics (computed by run.py --results).....39
Table 15: Performance evaluation.....39
Table 16: Agent execution per run (local Qwen2.5-3B).....40
Table 17: Results before and after the fixed missions.....40
Table 18: Sample prompts (full text in agents/prompts.py and dashboard/ask.py).....52
Table 19: Test data: flights in data/ (each with mission.waypoints, params.param, truth.json, meta.json).....53

CHAPTER 1 – INTRODUCTION

1.1 Introduction

Artificial Intelligence (AI) is the field of building systems that perform tasks normally requiring human intelligence, such as perception, reasoning and decision-making. **Generative AI** is the branch that produces new content; large language models (LLMs) such as Claude and Qwen are trained on large text corpora and generate text, code or structured data from a prompt. **Agentic AI** uses an LLM as the decision-making core of a system that pursues a goal over several steps: it plans, acts through tools or other agents, observes the results and adjusts [1], [11].

A traditional chatbot answers one message at a time from what the model already knows. An AI agent differs in six ways, each of which appears in AeroMind:

- **Autonomy** – after the user drops in a flight's files, AeroMind runs the whole analysis without further input.
- **Planning** – an orchestrator decides which agents to run, in what order and why, from what the data shows.
- **Reasoning** – each agent explains what happened and why, citing the metrics it relies on.
- **Tool usage** – Python tools parse the logs, compute metrics, predict battery use and edit the mission; the LLM never reads raw telemetry.
- **Memory** – every analysed flight is stored, so later analyses compare against the pilot's own baseline.
- **Goal-oriented execution** – the goal is a verified result: a verdict, causes, a risk score and a safer mission file, and work continues (with one self-correction round) until the result passes verification.

1.2 Background

Small unmanned aircraft running the open-source ArduPilot autopilot record their state as MAVLink telemetry. Ground stations such as Mission Planner save it as a `.tlog` file on every connection [8], [9]. A five-minute flight produces tens of thousands of messages describing attitude, position, battery, GPS, wind, vibration and the radio link.

Conventional tools, such as Mission Planner's log review, ArduPilot's UAV Log Viewer [10] and PX4 Flight Review [12], plot this data well but leave interpretation to the user. To know why a flight drifted the operator must know which message holds the cross-track error, what values are normal, how wind and speed interact, and how to turn that into a change to the mission. A more autonomous system can do this interpretation: compute the right metrics, relate them to causes, remember what is normal for this pilot, and propose concrete, bounded changes. Agentic AI fits because the analysis is a sequence of judgements that depend on what the data shows, while the arithmetic must stay exact. AeroMind therefore gives the arithmetic to tools and the judgement to agents, and checks the agents against the tools.

1.3 Problem Statement

The objective of this project is to develop an Agentic AI-based system that analyses the telemetry log, mission and parameters of an ArduPilot flight, explains how the flight and its telemetry link went, identifies what went wrong and why, scores the risk of the next mission and produces a corrected mission file. Existing log viewers plot the data but leave diagnosis, risk assessment and mission correction to an expert, and general-purpose chatbots invent numbers when asked about a log. Therefore, there is a need for a system that can reason over flight data like an expert while every statement it makes is checked against the data.

1.4 Motivation

- **Why this problem:** our team builds drones for the Smart India Hackathon, and we repeatedly lost time working out from logs why a flight behaved as it did.
- **Why automation:** the analysis is repetitive and needs expertise that most operators lack; a tool that does it in a minute after every flight changes how often it is done.
- **Why Agentic AI:** the steps depend on the data (a flight with no problems needs no root-cause analysis), and explanations and advice in plain language are what LLMs do well.
- **Who benefits:** student teams, hobbyists and small operators who fly ArduPilot without a dedicated flight-test engineer.
- **What motivated our design:** plotting tools leave interpretation to the user, and a stand-alone LLM can state numbers that are not in the data [6]; AeroMind combines the strengths of both.

1.5 Objectives

1. To design a multi-agent Agentic AI system that turns the files of one ArduPilot flight into a verdict, a flight story, a telemetry-link analysis, root causes, a next-mission risk score and a corrected mission.
2. To implement LLM-planned orchestration of specialist agents over a shared state, with a rule-checked plan and a rule-based fallback for every agent.
3. To integrate deterministic tools for log ingestion, metric computation, risk scoring and bounded mission editing, so that the LLM never computes a number.
4. To implement long-term memory of analysed flights for personal baselines, battery prediction, recurring issues and fix effectiveness.
5. To verify every agent claim against the data and to evaluate the system on simulated flights with injected faults and known ground truth.
6. To analyse the effectiveness and limitations of the system honestly, including the cases where it falls short of its targets.

1.6 Scope of the Project

- **Target users:** pilots and student teams flying ArduPilot multirotors.
- **Supported tasks:** post-flight analysis of one flight from `.tlog` files (several per flight are merged), an optional mission (`.waypoints`), parameters (`.param`) and pilot notes; questions about the flight; a corrected mission file.
- **Functional scope:** flight, battery, tracking, vibration, GPS, wind, attitude and telemetry-link metrics; phases and anomalies; five agents and a Verifier; flight history; dashboard, report and command line.
- **Technologies covered:** Python, pymavlink, pandas, Pydantic, Streamlit, Plotly, Jinja2, Claude or Ollama; ArduPilot SITL, MAVProxy and Gazebo Harmonic for the evaluation flights.
- **Expected usage:** after each flight, on a laptop, with or without an internet connection.
- **Boundaries:** post-flight analysis of ArduPilot `.tlog` files, evaluated on simulated flights; by design the system never uploads anything to the drone.

CHAPTER 2 – LITERATURE REVIEW

2.1 Existing Systems and Research

Table 1 summarises the work that shaped AeroMind: three agent frameworks, one self-correction method and the conventional flight-log tools used by drone operators today.

Table 1: Literature review summary

Sr. No.	Paper/System	Year	Approach	Key Finding	Limitation
1	ReAct, Yao et al. [1]	2023	Interleaves reasoning traces with tool actions in one LLM loop	Reasoning and acting together beat either alone on QA and decision tasks	The LLM chooses every action; a reasoning error propagates through the loop
2	AutoGen, Wu et al. [2]	2023	Multi-agent conversation between configurable LLM agents	Complex tasks decompose well into conversing specialist agents	Coordination is itself LLM-driven; no built-in check that outputs are grounded in data
3	MetaGPT, Hong et al. [3]	2024	Role-specialised agents following standard operating procedures with structured hand-offs	Structured intermediate outputs reduce errors between agents	Targets software engineering; roles are fixed by the framework
4	Reflexion, Shinn et al. [4]; Self-Refine, Madaan et al. [5]	2023	The model critiques its own output and retries with the feedback	Feedback loops improve results without retraining	When the critic is also an LLM, it can accept wrong answers
5	UAV Log Viewer [10], PX4 Flight Review [12], Mission Planner [9]	current	Plot flight logs; some automatic checks	Fast, reliable visualisation of every logged signal	Interpretation, root cause, risk and mission changes are left to the user

2.2 Comparative Analysis

Table 2 compares conventional flight-log tools with AeroMind on the features relevant to this project.

Table 2: Comparison of existing tools and the proposed system

Feature	Existing System (log viewers)	Proposed System (AeroMind)
Autonomous Execution	No: the user opens and reads each plot	Yes: files in, full analysis out
Task Planning	No	Orchestrator plans the agents from a manifest; plan checked by rules
Reasoning	No: the user interprets	Agents explain causes and risk, citing metrics

Feature	Existing System (log viewers)	Proposed System (AeroMind)
Tool Usage	Plotting only	Ingest, metrics, memory, risk rules, bounded mission editing
Memory	No	Flight history: baselines, battery prediction, recurring issues, fix effectiveness
RAG	No	No (not implemented; see Section 5.5)
Multi-Agent Support	No	Five specialist agents and a Verifier on a shared state
Human Intervention	Entirely manual	Pilot reviews the fixed mission before flying; nothing is uploaded

2.3 Research Gap

Agent frameworks [1]–[3] show how to decompose work across LLM agents, and self-refinement [4], [5] shows the value of feedback. They do not, however, guarantee that what an agent says is true, and hallucination remains the central risk of LLM output [6]. In a safety-relevant domain such as flight analysis, an invented battery figure or an unsafe mission edit is worse than no answer. Flight-log tools, on the other hand, are exact but explain nothing. The gap is a system that keeps the exactness of the tools and adds the explanation and planning of agents, with checks that are not themselves LLM judgements. AeroMind addresses it in four ways:

- Python computes every number, and the agents only see the computed metrics.
- A deterministic Verifier checks every claim against them.
- Python owns the safety-critical decisions: the risk score and the mission limits.
- Each agent's output is guaranteed to match its schema, and the analysis always completes.

CHAPTER 3 – PROPOSED SYSTEM

3.1 Proposed Solution

AeroMind is a pipeline of deterministic tools followed by a planned team of LLM agents, all working on one shared state. Its parts map onto the elements of an agentic system as follows:

- **User interaction:** the user uploads one flight's files in a Streamlit dashboard (or uses the `run.py` command line) and may add free-text pilot notes. Afterwards they can ask questions about the flight.
- **Agents and LLM:** five specialist agents, each a system prompt and a Pydantic output schema, call Claude through the Anthropic API [14] or a local Qwen2.5-3B model [13] through Ollama [15] with structured output.
- **Planning:** the orchestrator builds a manifest of what inputs exist and what looks wrong, asks the LLM for an ordered plan with a reason per agent, and corrects the plan with rules.
- **Reasoning:** each agent reasons over the metrics given to it and must cite the metric behind each claim.
- **Tools:** Python tools ingest the logs, compute metrics, keep history, score risk and edit the mission.
- **Memory and knowledge base:** `history.json` stores one record per analysed flight. The shared state is the working memory of one analysis. There is no document knowledge base and no retrieval (RAG is not used).
- **Execution:** agents run in plan order; the Verifier checks their outputs; an agent with more than 30 % of its claims rejected runs once more with the reasons.
- **Final response:** the approved results (rejected items replaced by rule-based ones) are shown in the dashboard and written to `report.html`, `mission_fixed.waypoints` and `changelog.md`.

3.2 System Requirements

3.2.1 Hardware Requirements

Table 3 lists the machine on which the system was developed and evaluated; all figures in this report come from it.

Table 3: Hardware requirements

Component	Requirement (as used)
Processor	AMD Ryzen 5 7535HS, 6 cores / 12 threads
RAM	16 GB installed (14 GB usable); analysis alone needs under 1 GB
Storage	About 25 MB for the project and demo flights; 3.3 GB for the optional local LLM
GPU	NVIDIA GeForce RTX 2050, 4 GB (optional: Gazebo rendering and faster local LLM inference)
Internet Connection	Optional: needed only for the Claude API and the map tiles

3.2.2 Software Requirements

Table 4 lists only the technologies actually used.

Table 4: Software requirements and technologies used

Software/Technology	Version/Specification
Operating System	Ubuntu 22.04.5 LTS
Programming Language	Python 3.10.12 (and Lua for the simulated battery)
IDE/Environment	Visual Studio Code 1.142; Git; pytest 6.2.5
Agent Framework	None: own orchestrator in Python with Pydantic 2.13 schemas (no LangChain/CrewAI)
LLM	Claude via Anthropic SDK 2.1.0 (claude-opus-5-5) or Qwen2.5-3B via Ollama 0.40.8
Vector Database	Not used
Embedding Model	Not used
Frontend	Streamlit 1.57.0, Plotly 6.7.0; Jinja2 3.1.6 for report.html
Data and parsing	pymavlink 2.4.49, pandas 2.3.3, NumPy 2.2.6, PyYAML 6.0.3
Simulation	ArduCopter SITL (ArduPilot 4.6.0-beta1 development build), MAVProxy 1.8.74, Gazebo Harmonic 8.15.0

3.3 Functional Requirements

Table 5 lists what the system must be able to do; Table 13 in Chapter 8 tests these requirements on real flights.

Table 5: Functional requirements

ID	The system shall…
FR1	Accept one or more .tlog files of a flight, an optional mission, parameters and pilot notes
FR2	Merge the logs by time and keep only the vehicle's messages
FR3	Compute flight, battery, tracking, vibration, GPS, wind, attitude and link metrics with statuses and a health score
FR4	Detect telemetry dropouts with time, duration, position and nearest waypoint
FR5	Plan which agents to run from the data, and run them on a shared state
FR6	Explain the flight, diagnose causes, score next-mission risk and recommend improvements
FR7	Verify every agent claim against the data and re-run an agent whose claims fail
FR8	Store each flight and use history for baselines, battery prediction and recurring issues
FR9	Apply mission edits only within hard limits and write a corrected mission and change log
FR10	Answer questions about the flight from verified data

ID	The system shall…
FR11	Always complete the analysis, even if an individual LLM call does not succeed

3.4 Non-Functional Requirements

- **Accuracy:** no number in the output may come from the LLM unchecked; metrics follow documented thresholds (ArduPilot guidance for vibration).
- **Reliability:** any failure of the LLM (timeout, invalid JSON, refusal, network error) falls back to rules; a circuit breaker skips an unreachable LLM server for five minutes.
- **Performance:** the tool stage takes under 1 s per flight; total time is set by the LLM agents' inference.
- **Scalability:** one flight per analysis; history grows by one small record per flight.
- **Security:** uploads are validated (extension, 200 MB size, safe file names); untrusted text is escaped before display; the dashboard listens on localhost only; API keys live in an untracked `.env` file.
- **Usability:** one page answers the main questions; other tabs hold the detail; saved runs replay offline.
- **Maintainability:** small modules (largest about 260 lines), typed data contracts, 236 automated tests.

CHAPTER 4 – AGENT ARCHITECTURE

4.1 Overall System Architecture

Figure 1 shows the architecture as implemented. Inputs pass through Python tools into a shared state. The orchestrator plans and runs the agents on that state. The Verifier filters their outputs, and only the approved set reaches the outputs. The approved causes and edits are written back to the flight history.

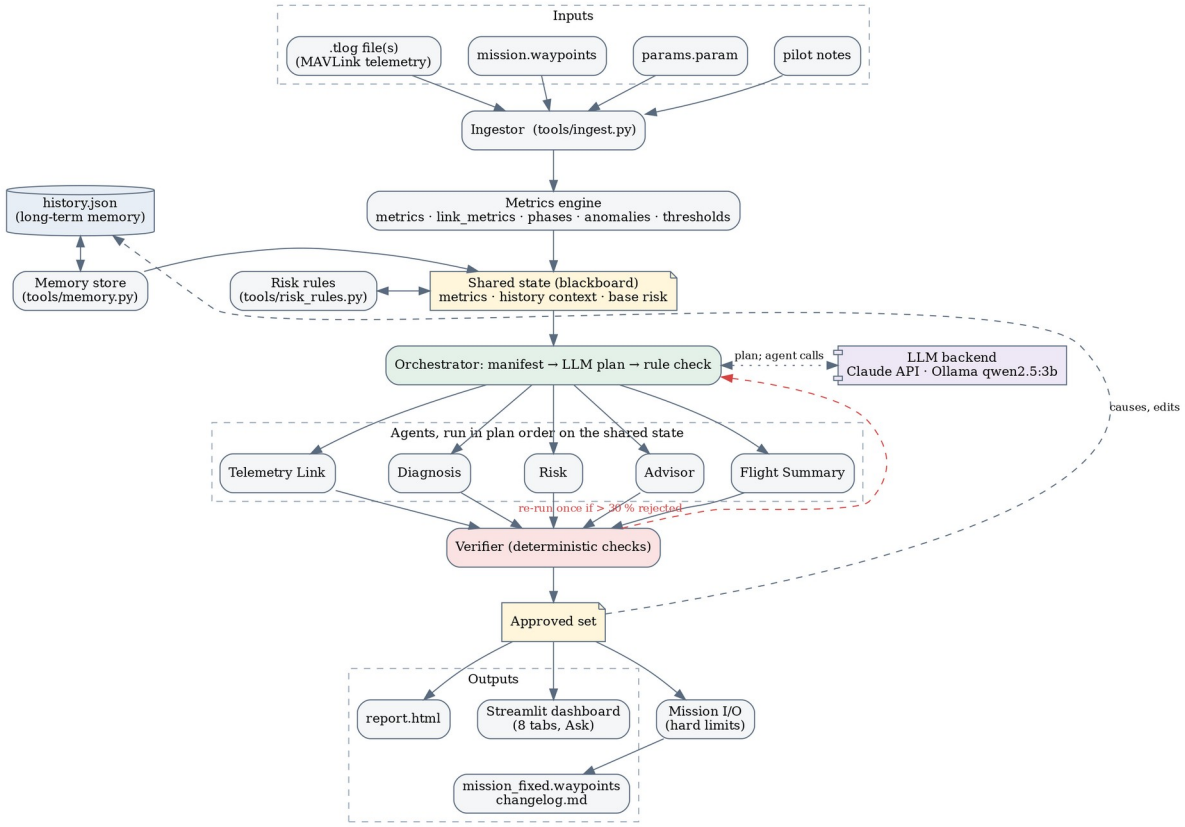


Figure 1: Overall system architecture of AeroMind

4.2 Architecture Components

Table 6 maps each component of a typical agent architecture to its implementation in AeroMind; components that are not used are marked as such.

Table 6: Architecture components

Component	Description (implementation)
User Interface	Streamlit dashboard `app.py` with 8 tabs and an Ask chat; command line `run.py`
LLM	`agents/llm.py`: ClaudeBackend (messages.parse with a Pydantic schema) or OllamaBackend (JSON schema)

Component	Description (implementation)
Agent	Five specialist agents, each a prompt (`agents/prompts.py`) and an output schema (`agents/schemas.py`), run by `run_agent()`
Planner	`agents/orchestrator.py`: manifest → LLM plan → `checked_plan()`; `default_plan()` if the call fails
Reasoning Module	The agents themselves, given only compact JSON context built by `agents/context.py`
Tools	`tools/`: ingest, metrics, link_metrics, phases, anomalies, risk_rules, memory, mission_io, report
Memory	`tools/memory.py` over `history.json`; the shared state dictionary within one analysis
Knowledge Base	Not used (no document store); domain rules live in `agents/hints.py` and the thresholds
Retriever	Not used
Executor	`orchestrator.run()` executes the plan; `mission_io.apply_edits()` executes mission changes
Output Generator	`approve()` builds the approved set; `pipeline.py` writes state.json, report.html, the fixed mission and change log

4.3 Agent Roles and Responsibilities

AeroMind is a multi-agent system. Table 7 lists the agents actually implemented; each has a rule-based fallback in `agents/fallbacks.py` that produces the same schema from the metrics alone.

Table 7: Agent roles and responsibilities

Agent	Role	Responsibility
Orchestrator	Planner and executor	Builds the manifest, gets and checks the plan, runs agents, triggers re-runs
Flight Summary	Narrator	Verdict (Good / Needs attention / Unsafe), a line per flight phase, 3-sentence summary
Telemetry Link	Link analyst	Findings on uptime, each dropout (time, duration, nearest waypoint), packet loss; whether the link was safe
Diagnosis	Analyst	Findings citing metrics; up to 3 root causes with category, confidence and supporting findings
Risk	Assessor	Adjusts the rule-based risk score by at most ±10 with a reason
Advisor	Planner of fixes	Mission edits tied to causes, from 4 allowed actions; top 3 improvements
Verifier	Reviewer	Deterministic: checks every claim, story line, number and edit mention against the data

4.4 Agent Workflow

The complete workflow of one analysis is shown in Figure 2.

1. The user provides the task: the files of one flight and optional notes (dashboard upload or command line).
2. The tools turn the files into metrics, statuses, phases, anomalies and a health score, so the objective is understood from data rather than from wording.
3. The memory store adds baselines, a battery prediction and recurring issues; the risk rules compute the base score.
4. The orchestrator creates the task plan: which agents run, in what order and why.
5. Each agent in turn receives only the context it needs and returns schema-checked JSON (or its fallback).
6. The Verifier observes the outputs and checks every claim against the data.
7. If more than 30 % of an agent's claims are rejected, that agent runs once more with the reasons.
8. The approved set is formed; mission edits are applied within the limits; history is updated.
9. The final result is presented in the dashboard and the report; the user can ask follow-up questions.

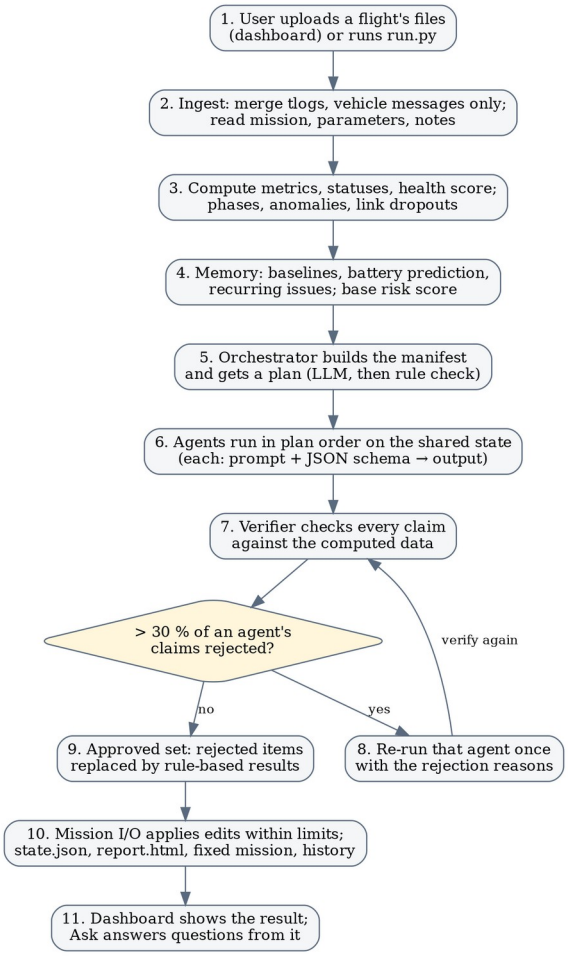


Figure 2: Agent workflow of one analysis

CHAPTER 5 – AGENTIC AI CONCEPTS USED

Only concepts that are implemented are described. Where a concept named by the template is not used, this is stated.

5.1 ReAct

AeroMind does **not** implement the ReAct loop [1], in which the LLM itself chooses a tool, observes the result and reasons again. This is deliberate: in AeroMind the tools always run, in a fixed order, before the agents. The agents never call tools, because a model that chooses tools could skip one or misread raw telemetry. The closest element is the verify–re-run cycle (Section 5.6): an agent's output is observed by the Verifier and the agent acts again with the feedback.

5.2 Planning

The orchestrator uses **plan-and-execute with a rule-checked plan**, shown in Figure 3. The planner sees a manifest (inputs present, metric statuses, which metrics are Caution or Bad, whether the link was healthy, flights in history). It returns an ordered list of agents with a one-line reason each. The plan is then corrected by `checked_plan()`:

- Flight Summary goes first and the Verifier last.
- Diagnosis runs only when some metric is Caution or Bad.
- Required agents are added, and duplicates are removed.
- The steps are put in dependency order.

If the call fails, `default_plan()` produces the plan from the same rules. The plan is dynamic in what it contains: a healthy flight skips diagnosis, and a healthy link gets a one-line rule result without an LLM call. In all eight evaluation runs the plan came from the LLM and was corrected by the rules.

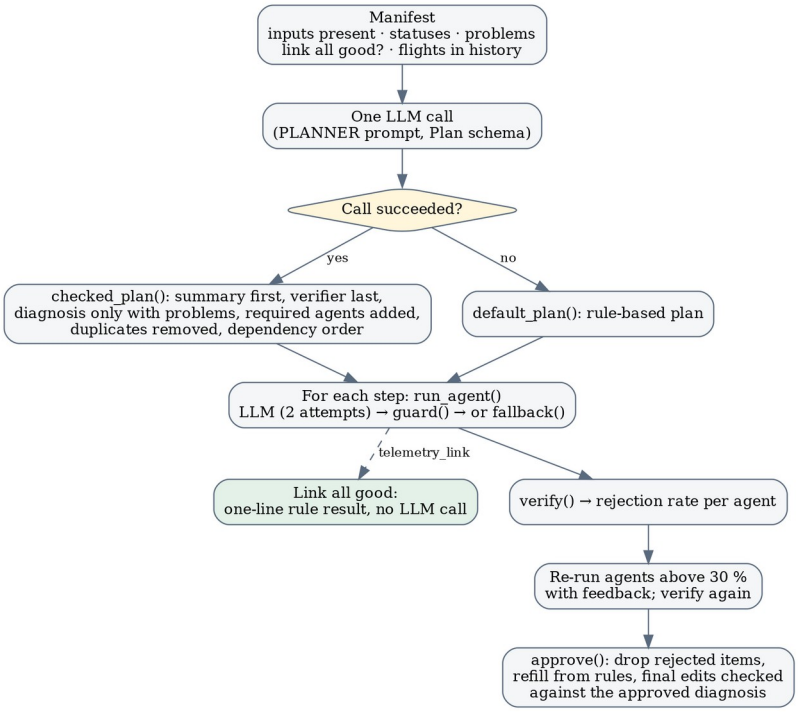


Figure 3: Planning and execution flow of the orchestrator

5.3 Tool Calling

Tools in AeroMind are deterministic Python functions invoked by the orchestration code, not by the LLM through function calling. Tool selection is therefore not an LLM decision. The pipeline always runs the analysis tools, and the plan decides which agents (and so which downstream tools such as mission editing) are used. Table 8 lists the tools, their inputs and outputs, and how the agents use their output.

Table 8: Tools: purpose, input and output

Tool	Purpose	Input	Output
Ingestor (ingest.py)	Read and merge logs, mission, parameters, notes	.tlog files, .waypoints, .param, text	FlightData: message tables, sequence numbers, file spans
Metrics engine (metrics.py, link_metrics.py, phases.py, anomalies.py, thresholds.py)	Every metric, status and the health score	FlightData	metrics.json (Pydantic Metrics)
Memory store (memory.py)	Baselines, battery prediction, recurrence, fix effectiveness	history.json, this flight	History context in the state
Risk rules (risk_rules.py)	Repeatable base risk score	metrics, parameters, prediction, recurrence	Score, level, factors, confidence
Mission I/O (mission_io.py)	Apply edits within limits	mission, proposed edits, causes	Fixed mission, applied and rejected edits
Report (report.py)	Printable report	approved state	report.html

Agents use tool output only as JSON context: the Diagnosis agent receives the statuses, metrics, anomalies and rule-derived candidate causes; the Risk agent receives the base score and factors; the Advisor receives the causes, the mission and valid suggested edits.

5.4 Memory

- **Short-term (working) memory:** the shared state dictionary of one analysis, a blackboard that every agent reads and adds to.
- **Long-term memory:** `history.json`, one validated record per analysed flight, which includes battery use per km, cross-track, link uptime, health, causes found and edits applied, plus the parent run when a flight re-flies a fixed mission. It is written atomically.
- **Conversational memory:** the Ask tab keeps the chat on screen for the session, but each question is answered independently; earlier turns are not sent to the LLM.
- **Vector-based memory:** not used.
- **Memory retrieval:** by computation, not similarity search: baselines need 3 or more flights, a 2σ test marks a metric worse than usual, the median mAh per km predicts battery, a cause in 3 of the last 5 flights is recurring, and re-flights are compared with their parent run.

5.5 Retrieval-Augmented Generation

RAG is **not implemented**. The agents' knowledge of the flight comes from computed metrics, not from retrieved documents. Retrieval over ArduPilot documentation for parameter advice is listed as future work (Chapter 11).

5.6 Multi-Agent System

- **Agent roles:** five specialist agents and a deterministic Verifier (Table 7).
- **Agent communication:** through the shared state (blackboard), not messages; each agent's validated output is added to the state and later agents read what they need (Risk reads the causes from Diagnosis; Advisor reads causes and risk).
- **Task delegation and orchestration:** the orchestrator delegates by plan; each agent call is `run_agent()` with two LLM attempts, a guard and a fallback.

- **Coordination:** Python guards keep shared quantities consistent. The risk score is always base plus adjustment. The link verdict follows the statuses. Final mission edits are checked against the approved diagnosis, not the raw one.
- **Execution flow and self-correction:** after all agents run, the Verifier computes a rejection rate per agent. Agents above 30 % re-run once with the rejection reasons, and the outputs are verified again.

```
    def run_agent(name: str, state: dict, outputs: dict, backend, fallback,
    mission=None,
                  feedback: Optional[str] = None) -> AgentResult:
        """The agent's output: from the LLM when it answers validly, otherwise
    from `fallback()`."""
        started = time.monotonic()
        errors = []
        if backend is not None:
            user = context.to_prompt(context.build(name, state, outputs, mission))
            if feedback:
                user += f"\nYour previous answer was rejected by the verifier:
    {feedback}. Fix exactly these points."
            for _ in range(ATTEMPTS):
                try:
                    answer = backend.complete(prompts.SYSTEM_PROMPTS[name], user,
    AGENT_OUTPUTS[name])
                    output = guard(name, answer, state, bool(mission))
                    return AgentResult(name, output, "llm", round(time.monotonic()
    - started, 2), tuple(errors), feedback)
                except (LLMError, ValidationError) as error:
                    errors.append(str(error)[:300])
                except Exception as error:  # an unexpected backend failure must
    not stop the analysis
```

Listing 1: run_agent() – LLM with two attempts, guard, then the rule-based fallback (agents/base.py).

5.7 Human-in-the-Loop

AeroMind keeps the human in control at the point of action rather than through an approval step inside the application. Nothing is ever uploaded to the vehicle. The pilot downloads `mission_fixed.waypoints`, reviews it and the change log in a ground station, and decides whether to fly it. A rejected change has no effect. The pilot also contributes input through free-text notes, which the agents use only as supporting evidence. An in-app approve/reject workflow was not implemented.

CHAPTER 6 – METHODOLOGY

6.1 Input Processing

Files arrive by dashboard upload (validated extensions, 200 MB limit, sanitised names, unique run name) or by the command line. `tools/ingest.py` reads each `.tlog` with pymavlink [7], [16]. It identifies the vehicle from its HEARTBEAT, keeps only the vehicle's messages and merges several files of one flight by timestamp. The time span of each file is kept, so a gap between files counts as a dropout. Parameters are read in Mission Planner or MAVProxy format, and notes are cleaned and capped at 2,000 characters.

6.2 Task Understanding

The objective is always the same – explain this flight and make the next one safer – so task understanding means understanding the data. The metrics engine computes about 30 metrics and rates 14 of them Good, Caution or Bad (Table 9). The health score starts at 100 and loses 10 per Caution and 25 per Bad. Two measurement decisions came from observing real logs:

- Vibration and attitude are measured only while airborne, because the landing touchdown alone produced 37 m/s² and one accelerometer clip on a calm flight.
- A dropout must be silence on both the ground-station clock and the vehicle's own clock (`time_boot_ms`), because a simulator running slower than real time showed only 4.4 s of vehicle time in a 10 s wall-clock window.

Table 9: Metrics and thresholds

Metric	Good	Caution	Bad
Cross-track error, max (m)	< 1	1–3	> 3
Battery at landing (%)	> 30	20–30	< 20
Vibration max, airborne (m/s²)	< 30	30–60	> 60
Accelerometer clipping	0	any increase	–
Minimum satellites / maximum HDOP	≥ 10 / < 1.5	6–9 / 1.5–2.5	< 6 / > 2.5
Wind estimate, max (m/s)	< 5	5–8	> 8
Roll / pitch, max (°)	< 25	25–35	> 35
Mission completion (%)	100	–	< 100
Warnings and failsafes	0	1–2	> 2
Link uptime (%)	≥ 99	95–99	< 95
Dropouts (silence > 2 s)	0	1–2, each ≤ 10 s	> 2, or any > 10 s
Packet loss (%)	< 2	2–5	> 5
Position update rate	steady	–	worst 10 s < 50 % of median

6.3 Task Planning

The analysis is divided into fixed tool steps and planned agent steps. The tool steps always run: ingest, metrics, memory, base risk. The planned steps are the agents the orchestrator selects (Section 5.2), each a sub-task with one schema: summarise, analyse the link, diagnose, assess risk, advise. Splitting the work this way keeps each prompt short, which matters for a 3B local model.

6.4 Reasoning and Decision Making

Each agent decides within a narrow remit. Diagnosis receives candidate causes from documented hint rules (for example, wind Caution/Bad together with cross-track or attitude Caution/Bad points to crosswind drift). It may reword them and adjust confidence. The Risk agent may move the rule score by at most ±10 and must give a reason; a reduction must name what lowers the risk. Python keeps the decisions that must be exact. The rule-based risk factors are listed in Table 10.

Table 10: Base risk rules (Low < 30, Medium 30–59, High ≥ 60, capped at 100)

Risk factor	**Points**
Predicted battery at landing < 10 % / < 20 % / 20–30 %	+45 / +30 / +15
Battery at landing Bad in the latest flight	+15
Telemetry dropouts or uptime Bad / Caution	+20 / +10
GPS Bad / Caution	+20 / +10
Wind Bad in the latest flight	+15
Vibration Bad	+15
Each failsafe not set (FS_THR_ENABLE, BATT_FS_LOW_ACT, FENCE_ENABLE = 0)	+10
A recurring issue from history	+10

6.5 Tool Selection

Tools are selected by the pipeline, not by the LLM (Section 5.3). Agent selection is planned: the manifest decides whether Diagnosis runs, and whether the Telemetry Link agent needs the LLM or a one-line rule result. Mission I/O runs only when a mission file was given.

6.6 Task Execution

`orchestrator.run()` executes the plan. Every LLM call requests structured output against the agent's schema. An invalid reply is retried once with the validation error. A second failure,

or any unexpected error, produces the rule-based output, so execution always completes. Mission edits are executed by `mission_io.apply_edits()` within the limits in Table 11.

Table 11: Mission edit actions and limits

Action	How it is applied	Limits
set_speed	Changes or inserts DO_CHANGE_SPEED before the waypoint	3–10 m/s
change_altitude	Changes the waypoint's altitude	±20 m per edit; final 25–120 m
remove_waypoint	Deletes a navigation waypoint	≤ 30 % of waypoints; never home, take-off, RTL or land
add_hold	Sets the hold time at a waypoint	0–10 s
All edits	Each tied to a diagnosed cause; at most 5	Mission must end in RTL/LAND; no leg may pass the 44 m radio mast lower than its top plus 6 m

The 25 m floor replaces the PRD's 10 m because the evaluation world has 16.4 m buildings and 9 m power lines; the 120 m ceiling matches the green-zone limit of India's Drone Rules, 2021 [17].

6.7 Observation and Feedback

The Verifier observes every output. A claim must cite a metric path that exists, and its value must match within 5 % (or 0.15 absolute). Every number with a unit in any text (story lines, summary, causes, reasons, recommendations) must match a computed value **of the same unit**. Word units such as seconds, knots and km/h are converted first. Story statuses must equal the data's status for that phase. A cause may not rest on a rejected finding. A recommendation may only name edits that passed the limits. The rejection reasons are the feedback for the re-run.

```
    def check_text(text: str, known: dict, require_number: bool = False, where:
    str = "the flight data") -> str:
        """'' if every number with a unit in `text` matches a known value of that
    unit."""
        if require_number and not BARE_NUMBER.search(text):
            return "gives no number from the flight data as evidence"
        for raw, unit in NUMBER_WITH_UNIT.findall(text):
            family, factor = UNITS.get(unit.lower(), UNITS.get(unit, (None, 1)))
            value = float(raw) * factor
            if family is None or not any(close(value, k, family) for k in
    known.get(family, [])):
                return f"'{raw} {unit}' is not in {where}"
        return ""
```

Listing 2: unit-aware grounding of numbers in free text (agents/verifier.py).

6.8 Final Response Generation

`approve()` builds the final response. Rejected story lines are replaced by the data's own line, a rejected summary by the rule-based one, rejected findings and causes are dropped, and recommendations are refilled from the rules to three. Final mission edits are re-derived against the approved diagnosis. `pipeline.py` then writes `state.json` (everything, replayable without an LLM), `report.html`, `mission_fixed.waypoints` and `changelog.md`, and stores the approved causes and edits in history.

6.9 Evaluation Data: Simulated Flights with Known Faults

To evaluate against ground truth, flights were flown in ArduCopter SITL [8] inside the team's Gazebo [18] disaster world (Figure 4, Figure 5). The world and a Hexa-X hexacopter are vendored into the repository with their licences. The simulator lacked two things the faults needed, and both were added:

- **Battery:** the Gazebo JSON backend simulates no battery, so a Lua script feeds ArduPilot's scripting battery monitor with current derived from the motor outputs.
- **Wind:** ArduPilot's wind parameters do not move a Gazebo vehicle, so wind is a constant force on the airframe. Because the airframe's drag is linear, that force is exactly equivalent to wind.

Link faults are injected by AeroMind's own recording ground station, which writes the true outage times to `truth.json`. Table 12 lists the scenarios.

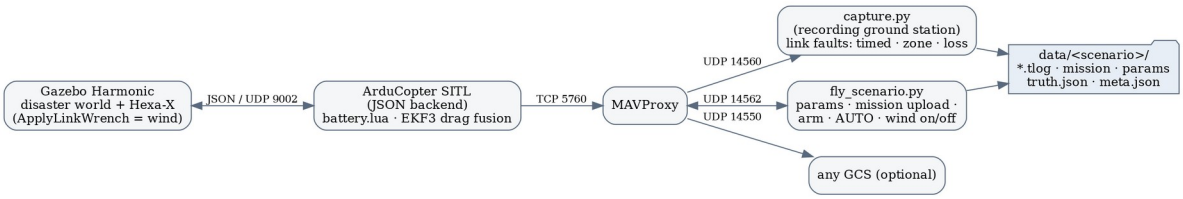


Figure 4: Simulation and data-collection pipeline

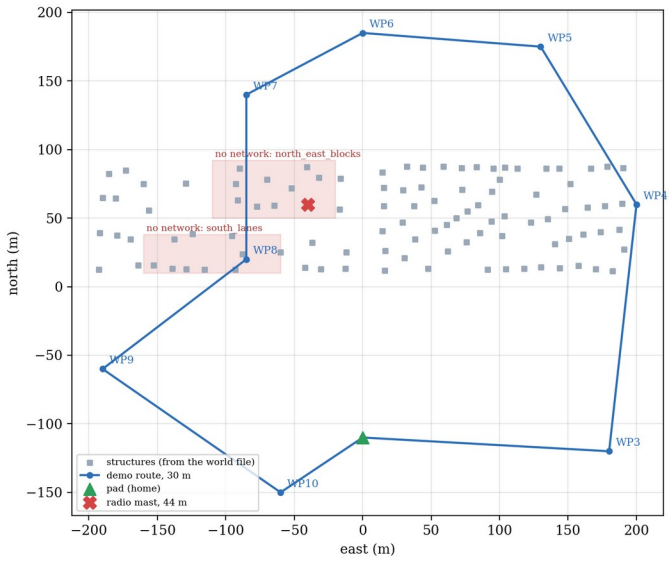


Figure 5: Demo route in the disaster world (structures from the world file)

Table 12: Evaluation scenarios

ID	Scenario	How the fault is injected
S0	Baseline, 5 m/s	None
S1	Strong wind + fast flight	9 N force (10 m/s from the east) above 3 m; mission speed 12 m/s
S2	Undersized battery	BATT_CAPACITY 2200 mAh instead of 10000
S3	Telemetry dropout	Recording link down for 12 s at 90 s and at 200 s after arming
S4	Combined	S1 + S2 + link down inside the north-east no-network zone
S1/S2/S4 fixed	Re-flights	Same faults, flying AeroMind's mission_fixed.waypoints

CHAPTER 7 – IMPLEMENTATION

7.1 Project Structure

```
AeroMind/
├── app.py                  Streamlit dashboard (entry point)
├── run.py                  command line: single flight, --batch, --compare,
--results
├── pipeline.py             one analysis end to end, writes outputs/<run>/
├── agents/                 schemas, llm (Claude/Ollama), prompts, context,
base (run_agent),
│                           orchestrator, verifier, hints, fallbacks
├── tools/                  ingest, metrics, link_metrics, phases, anomalies,
thresholds,
│                           timeline, memory, risk_rules, mission_io,
waypoints, site,
│                           geo, analysis, report, results
├── dashboard/              summary_tab, detail_tabs, charts, data, ask, text
├── report/template.html    report.html template (Jinja2)
├── sim/                    world, models, SITL launcher, capture, scenarios,
battery.lua
├── data/<scenario>/        flown logs, mission, params, truth.json, meta.json
├── outputs/<run>/          state.json, report.html, mission_fixed.waypoints,
changelog.md
├── history.json            long-term flight memory
└── tests/                  236 pytest tests
```

Listing 3: actual project structure (about 5,000 lines of Python).

7.2 Agent Implementation

Each agent is a system prompt plus a Pydantic schema. Listing 4 shows the Diagnosis prompt. The shared grounding rule appended to every prompt tells the model to copy numbers exactly and to say when a metric is null. Listing 5 shows the guard that keeps the risk score Python's, whatever the model returns.

```
DIAGNOSIS = f"""You are a drone failure analyst. List findings that cite
metric names and values
(id F1, F2, ...; metric is a dotted path into the metrics, value is its
number). Then give up to 3 root
causes (id C1, C2, ...) with a category, a one-sentence cause, confidence 0-1
and the finding ids that
support it. Start from `candidate_causes`, which the rules already found; keep
their categories,
improve the wording and adjust confidence if the evidence warrants. Use pilot
notes as supporting
evidence only.
{HINT_TEXT}
{GROUNDING}"""
```

Listing 4: the Diagnosis agent's system prompt (agents/prompts.py).

```
def guard(name: str, output: BaseModel, state: dict, has_mission: bool) ->
BaseModel:
    if name == "risk":
        base = state["risk_base"]
        score = max(0, min(risk_rules.MAX_SCORE, base["score"] +
output.adjustment))
        return output.model_copy(update={
```

```
                "risk_score": score, "risk_level": risk_rules.level(score),
    "factors": [f["factor"] for f in base["factors"]],
                "predicted_battery_pct": state["history"]["battery_prediction"]
    ["predicted_end_pct"],
                "confidence": base["confidence"]})
            if name == "telemetry_link":
                from agents import hints
                verdict = hints.worst(state["metrics"]["status"], "dropouts",
    "link_uptime", "packet_loss", "position_rate")
                return output.model_copy(update={"link_verdict": verdict,
    "safe_for_mission": verdict != "bad"})
```

Listing 5: guard() – Python owns the risk score and the link verdict (agents/base.py).

The LLM backend requests structured output. With Claude, `client.messages.parse()` validates the reply against the Pydantic model [14]. With Ollama, the model's JSON schema is passed as the `format` and the reply is validated with Pydantic [19].

7.3 Tool Implementation

Link metrics (tools/link_metrics.py)

- **Purpose:** Find dropouts and measure link quality across the whole flight
- **Input:** All vehicle messages with receive time and MAVLink sequence number; the vehicle clock
- **Output:** Uptime, dropouts (time, duration, position, nearest waypoint, between files), packet loss, RSSI, position rate
- **Implementation:** Gaps longer than 2 s in receive time that are also longer than 2 s on the vehicle clock; packet loss from sequence-number gaps (mod 256) excluding pairs that span a dropout
- **Role in agent workflow:** Feeds the Telemetry Link agent, the link strip and the risk rules

Mission I/O (tools/mission_io.py)

- **Purpose:** Turn proposed edits into a safe mission file
- **Input:** Mission items, proposed edits, approved cause ids
- **Output:** Fixed mission, applied and rejected edits, change log
- **Implementation:** Each edit checked against Table 11, applied to a copy, then the whole mission re-checked for RTL/LAND and obstacle clearance
- **Role in agent workflow:** Executes the Advisor's plan; the only path from agents to a mission file

7.4 Memory Implementation

- **Memory type:** long-term episodic memory of flights, plus the per-analysis shared state.
- **Storage mechanism:** `history.json`, a list of validated `HistoryEntry` records, written atomically (temporary file and rename) so a crash never leaves a half-written history.
- **Data stored:** run id, date, scenario, parent run, mission length, duration, battery use (% and mAh per km), capacity, battery at landing, completion, cross-track, uptime, dropouts, vibration, wind, health score, cause categories and applied edits.
- **Retrieval mechanism:** computed statistics over the stored records (Section 5.4).
- **How memory influences decisions:**

- The battery prediction drives the largest risk factor.
- Recurring causes add risk.
- Worse-than-usual flags appear on the Summary.
- The Advisor is shown which past edits improved which metrics.

7.5 RAG Implementation

Not applicable: RAG is not implemented (Section 5.5).

7.6 User Interface

The Streamlit dashboard [20] is the main interface. Figure 6 shows the home page with the input form in the sidebar (telemetry logs, mission, parameters, pilot notes, run name) and the Summary tab of scenario S4. The other tabs hold the detail:

- **Agents** (Figure 7): the agents' execution, with the orchestrator's plan, each agent's source and time, and every Verifier verdict.
- **Ask** (Figure 8): questions answered from the verified data.
- **Track, Telemetry and History** (Figures 10–12, Appendix B): map and link views, and baselines, trends and what worked before.

All screenshots were captured from the running application by `docs/report/screenshots.py`.

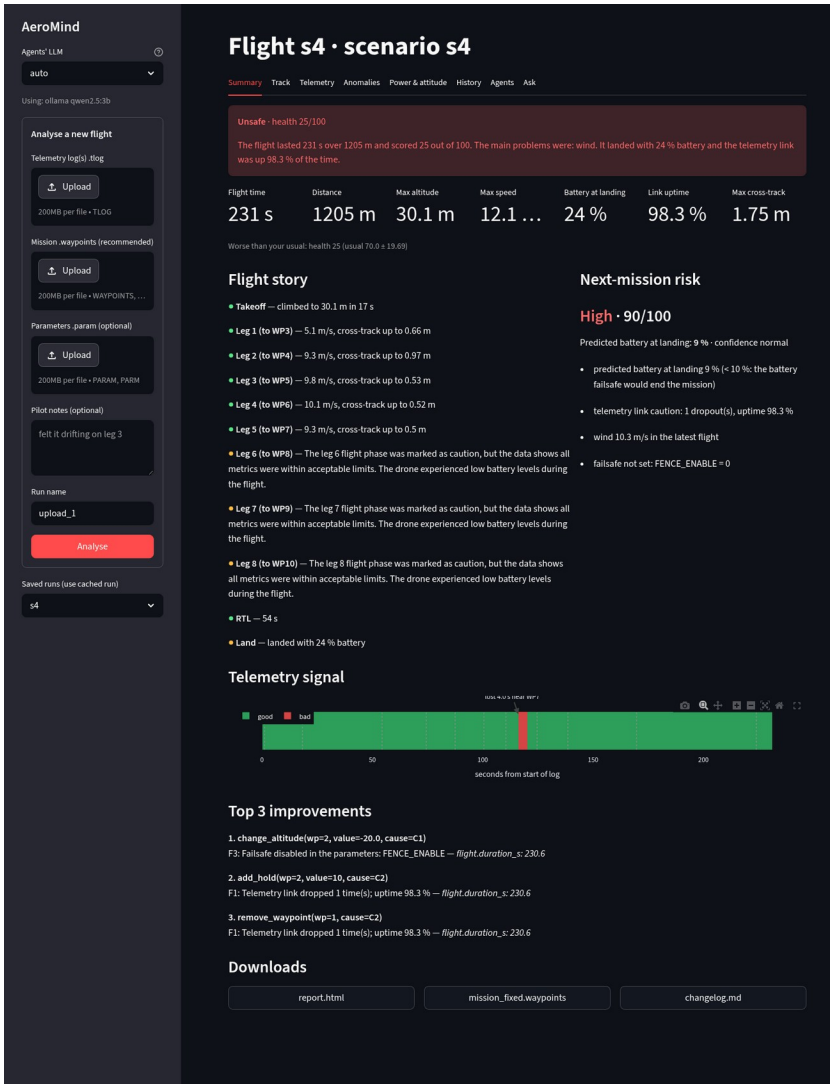


Figure 6: Home page: input form (sidebar) and the Summary tab for S4

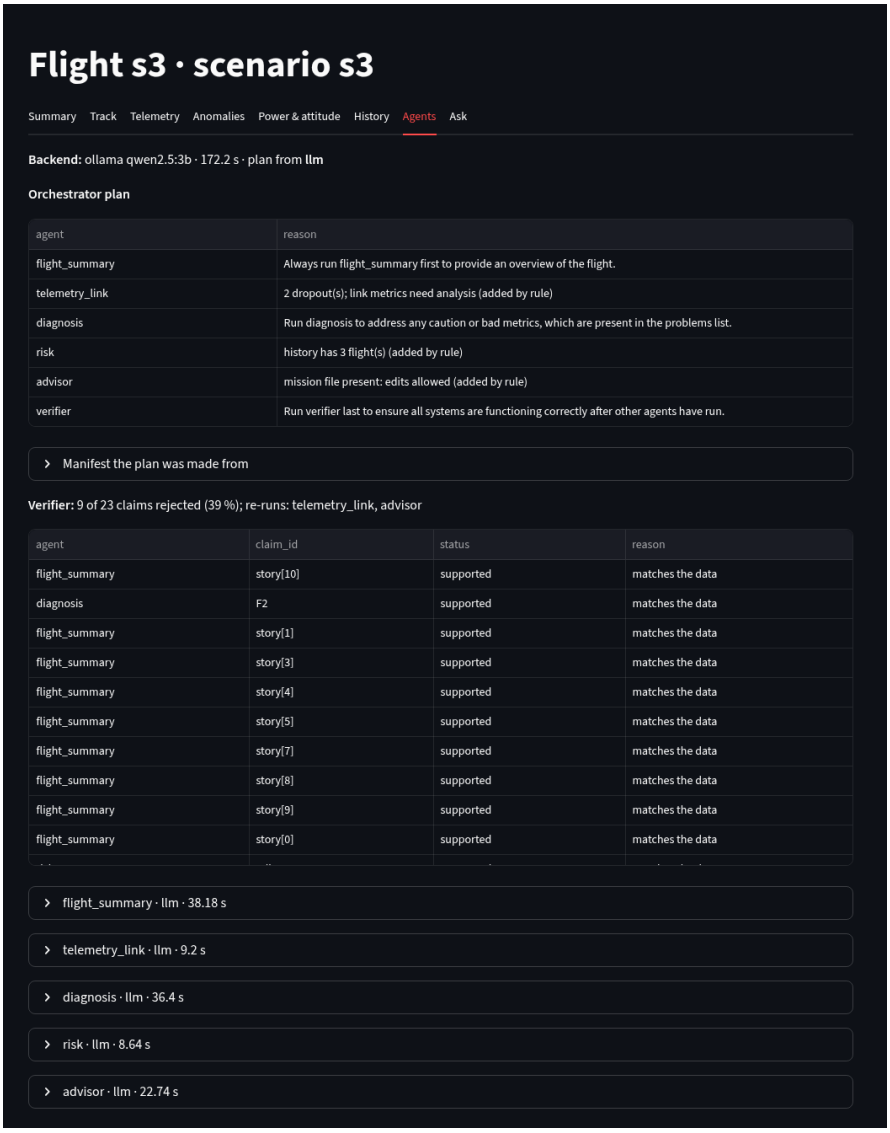


Figure 7: Agents tab (S3): orchestrator plan, Verifier verdicts and each agent's execution

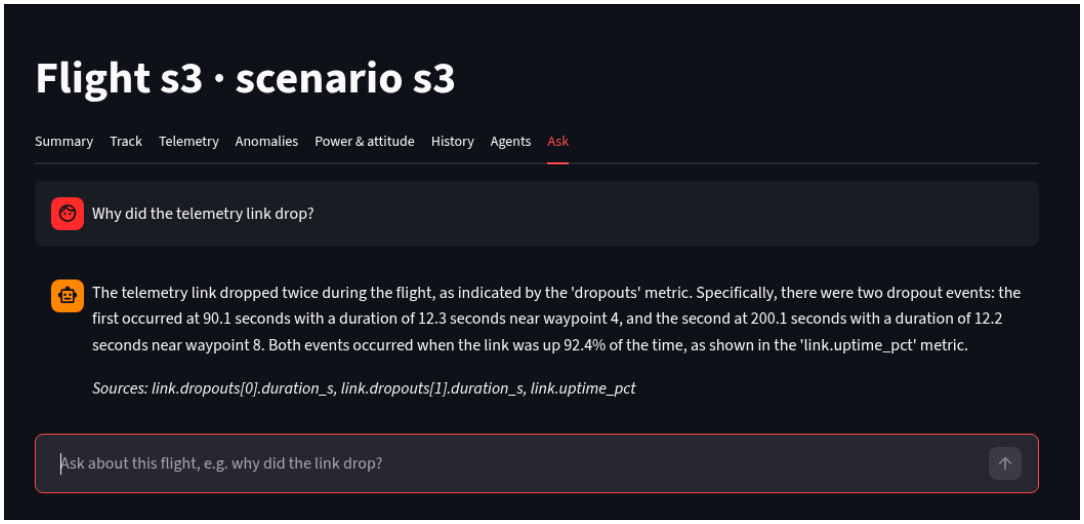


Figure 8: Ask tab (S3): an answer from the local LLM, verified against the cited metrics

CHAPTER 8 – RESULTS AND OUTPUT

8.1 Test Cases

Table 13 lists the scenario test cases. Every actual output is taken from the saved runs in `outputs/`, and the ground truth for the dropouts comes from `data/s3/truth.json`.

Table 13: Test cases and actual results

ID	Input	Expected Output	Actual Output	Status
TC01	S0 baseline flight	Good verdict, no Bad metric, Low risk	Good, health 100, risk Low 10	Pass
TC02	S1 wind + 12 m/s	Wind flagged; wind cause; speed edit	wind 10.3 m/s Bad; causes: wind_drift, setup; edit: set_speed WP3 6 m/s	Pass
TC03	S2 small battery	Battery Bad; low_battery; shorten route; High risk	landed 14 %; causes: low_battery, setup; remove_waypoint WP5; risk High 60	Pass
TC04	S3 two 12 s link outages	2 dropouts within 3 s of truth, nearest waypoint	dropouts at 90.1 s and 200.1 s (truth 90.2 s, 200.2 s), near WP4 and WP8	Pass
TC05	S4 combined faults	Wind, battery and link causes; High risk	causes: link_loss, wind_drift, setup; risk High 90	Pass
TC06	S2 re-flight with fixed mission	Battery at landing ≥ 25 %	25 %	Pass
TC07	S1 re-flight with fixed mission	Max cross-track ≥ 40 % lower	1.8 m → 1.44 m (20 % lower)	Partial
TC08	Ask: “Why did the telemetry link drop?” (S3)	Answer citing link metrics, verified	LLM answer citing 3 metric paths, passed verification (Figure 8)	Pass
TC09	Invented number in a cause (unit test)	Verifier rejects the cause	“Motor 3 failing at 2400 rpm and 77 m/s gusts” rejected	Pass
TC10	End-to-end time with the local LLM	< 60 s	median 174 s	Partial

8.2 Sample Outputs

Test Case 1 – combined faults (S4)

- **Input:** The S4 logs, mission and parameters, with the note “felt it drifting on leg 3”
- **Agent action:** The orchestrator's LLM plan ran flight summary, telemetry link, diagnosis, risk, advisor. The Verifier checked 26 claims and the flight summary, telemetry link, risk, advisor agent(s) refined their answers in the self-correction round.
- **Output:** verdict Unsafe; causes link_loss, wind_drift, setup; risk High 90. Fixed mission: set_speed WP3 6 m/s.

Test Case 2 – undersized battery (S2)

- **Input:** The S2 logs, mission and parameters
- **Agent action:** The orchestrator's LLM plan ran flight summary, telemetry link, diagnosis, risk, advisor. The Verifier checked 24 claims and the diagnosis, advisor agent(s) refined their answers in the self-correction round.
- **Output:** verdict Needs attention; causes low_battery, setup; risk High 60. Route 1330 m → 1301 m; report.html written (Figure 9).

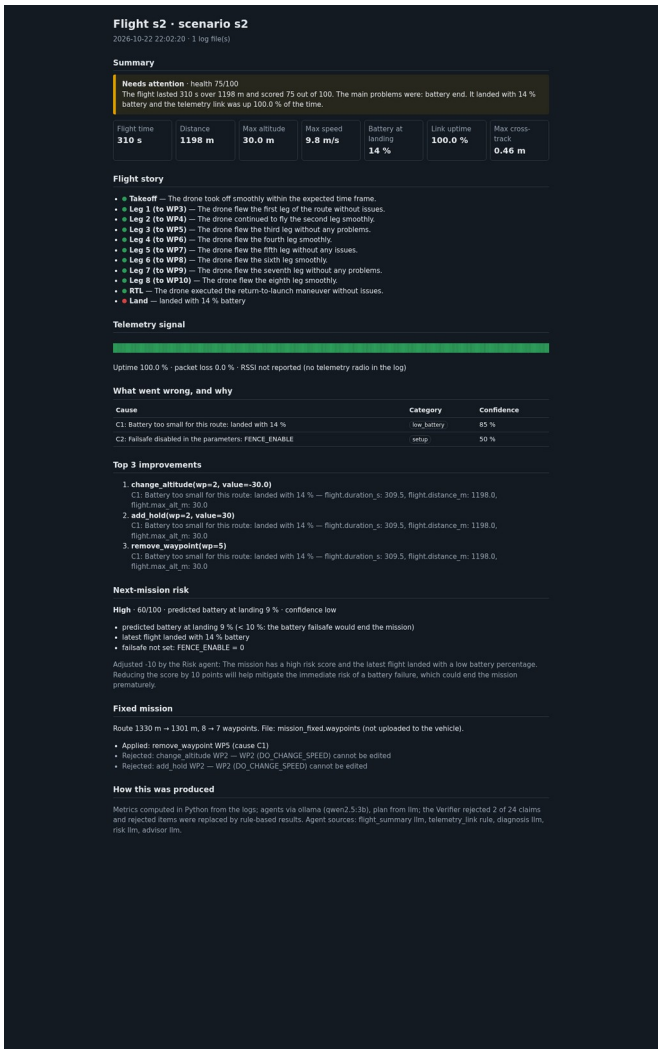


Figure 9: Generated report.html for S2

8.3 Agent Execution Trace

The execution trace of S3, taken from `outputs/s3/state.json`:

```
   User query      S3 logs + mission + params (two injected 12 s link outages)
      ↓
   Understanding   health 50; Bad: link_uptime (92.4 %), dropouts (2)
      ↓
   Plan (LLM)      flight_summary → telemetry_link → diagnosis → risk → advisor →
```

```
   verifier
      ↓
   Execution      flight_summary llm 38.18 s, telemetry_link llm 9.2 s, diagnosis
   llm 36.4 s, risk llm 8.64 s, advisor llm 22.74 s
      ↓
   Observation    Verifier: 23 claims, 9 rejected
      ↓
   Decision       re-run: telemetry_link, advisor
      ↓
   Final response Needs attention; causes: link_loss, setup; risk Low 25;
                  first advice: Raise the ground antenna and keep the route
   within radio range (first loss near WP4)
```

Listing 6: execution trace of S3 (agent times are those of the final, re-run call).

CHAPTER 9 – PERFORMANCE EVALUATION

All values in this chapter were measured on the hardware in Table 3. Table 14 gives the project's success metrics exactly as computed by `python3 run.py --results` (docs/RESULTS.md).

Table 14: Success metrics (computed by run.py --results)

Metric	Measured	Target	Result
Fault detection	4 of 4 (s2, s1, s3, s4)	>= 3 of 4	pass
Root-cause accuracy	4 of 4 (top: s2 low_battery, s1 wind_drift, s3 link_loss, s4 link_loss)	>= 3 of 4	pass
Dropout localisation	0.1 s off, 0.1 s off	both S3 dropouts within 3 s	pass
False alarms	0 Bad on S0	0	pass
Tracking improvement	S1 max cross-track 1.8 m → 1.44 m (20% lower)	>= 40 % lower	fail
Battery margin	S2 fixed landed with 25 %	>= 25 %	pass
Battery prediction error	s1: predicted 80 %, actual 81 %; s2: predicted 31 %, actual 25 %; s4: predicted 31 %, actual 24 %	within 10 points	pass
Risk ranking	s0 Low, s2 High, s4 High	S0 Low; S2, S4 High	pass
Grounding	55 of 185 claims rejected (30 %) via ollama qwen2.5:3b	report the rate	n/a
Speed	files in → dashboard ready: max 235 s, median 174 s	< 60 s	fail

Table 15 evaluates the system against the metrics suggested for agentic systems; retrieval accuracy does not apply because no retrieval is used.

Table 15: Performance evaluation

Metric	Evaluation Method	Result
Task Completion Rate	Analyses completed / analyses started	8 / 8 (100 %)
Accuracy (fault detection)	Injected faults flagged / injected faults (S1–S4)	4 / 4
Accuracy (root cause)	Top cause matches the injected fault (S1–S4)	4 / 4
Tool Success Rate (Python tools)	Analyses whose tools completed / total	8 / 8
Agent success rate	Agent results produced by the LLM agents / agent runs	34 / 34 (100 %)
Average Response Time	Files in → results saved (Qwen2.5-3B on the laptop's CPU (AMD Ryzen 5 7535HS))	median 174 s, range 114–235 s
Tool-stage time	Ingest, metrics, memory and risk rules only (S0–S4)	0.70–0.97 s
Grounding	Claims checked by the Verifier; refined in self-correction	185 checked; 55 refined (30 %)
Retrieval Accuracy	Relevant retrievals / total	Not applicable (no RAG)
Test Case Success Rate	Scenario test cases passed (Table 13)	8 / 10

Metric	Evaluation Method	Result
Automated tests	pytest: passed / total	236 / 236
Code coverage	Lines covered (agents, tools, dashboard, pipeline)	91 % (78 % including SITL drivers)

Table 16 shows per-run agent behaviour, including the agents whose answers were refined in the self-correction round.

Table 16: Agent execution per run (local Qwen2.5-3B)

Run	Plan	LLM agents	Claims verified	Agents refined by self-correction	Total s
s0	llm	3	19	advisor	114
s1	llm	4	24	flight summary, advisor	175
s2	llm	4	24	diagnosis, advisor	184
s3	llm	5	23	telemetry link, advisor	173
s4	llm	5	26	flight summary, telemetry link, risk, advisor	235
s1_fixed	llm	4	23	flight summary, diagnosis, advisor	170
s2_fixed	llm	4	21	diagnosis, advisor	153
s4_fixed	llm	5	25	flight summary, telemetry link, advisor	174

Finally, Table 17 compares each scenario before and after its fixed mission was flown under the same faults.

Table 17: Results before and after the fixed missions

Scenario	Verdict	Health	Risk	Key metric	Edits flown
s0	Good	100	Low 10	health: 100	–
s1	Needs attention	55 → 55	Medium 35 → Medium 45	max cross-track m: 1.8 → 1.44	speed 12 → 6 m/s
s2	Needs attention	75 → 90	High 60 → Medium 30	battery at landing %: 14.0 → 25.0	removed WP3, WP4
s3	Needs attention	50	Low 25	link uptime %: 92.4	–
s4	Unsafe	25 → 25	High 90 → Medium 55	battery at landing %: 24.0 → 24.0	removed WP3, WP4; speed 12 → 6 m/s

Interpretation. The agents met the detection, root-cause, dropout-localisation, false-alarm, battery-margin, battery-prediction and risk-ranking targets. The mission they corrected for S2 raised the battery at landing from 14 % to 25 % and lowered the risk from High to Low. For S1 the speed fix reduced the maximum cross-track error from 1.8 m to 1.44 m; the improvement is smaller than the 40 % target because the original error was already modest (the airframe's tilt limit capped its speed at about 9.5 m/s). End-to-end time with the local 3B model was a median of 174 s, almost all of it LLM inference on a the laptop's CPU (AMD

Ryzen 5 7535HS), since the tool stage takes under 1 s; GPU inference or the Claude API shortens it directly.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

The main challenges met during development, and how each was resolved, are summarised below.

Keeping LLM output grounded. A small model sometimes stated numbers or statuses that were not in the data. The deterministic Verifier catches these, and the self-correction round lets the agent refine its answer, so only verified content reaches the user.

Consistent agent decisions. Early runs showed the Risk agent lowering scores without a reason. A rule now requires a lowered score to name what reduces the risk, and Python owns the final score (base ± 10).

Response time. Several LLM calls on a laptop dominate the run time. Agents receive compact JSON context, the Telemetry Link agent is skipped when the link is healthy, and a circuit breaker avoids waiting on an unresponsive model server.

Simulating the faults. SITL over Gazebo simulated no battery and ignored ArduPilot's wind parameters. We added a Lua battery model driven by the motor outputs and modelled wind as an equivalent force on the airframe, applied above 3 m so that landings disarm normally.

Accurate measurement. The landing touchdown jolts the IMU, and a simulator running slower than real time produces gaps in receive times. Vibration is therefore measured while airborne, and a dropout must also appear on the vehicle's own clock.

Code quality and security. An independent code review was carried out and every finding was fixed with a regression test. Uploads are validated, displayed text is escaped, the dashboard listens on localhost and the LLM never sees raw files.

Scope. The system was evaluated on eight simulated flights, and it reads `.tlog` telemetry logs. Real-flight logs and other log formats are natural next steps (Chapter 11).

CHAPTER 11 – FUTURE SCOPE

- **Larger models:** run the agents on Claude and compare speed and answer quality with the local 3B model.
- **Real flights:** analyse real-drone logs with telemetry-radio RSSI, and read DataFlash `.bin` logs (pymavlink already reads them).
- **Richer verification:** add semantic checks for statements that contain no numbers.
- **Live mode:** stream telemetry during flight and raise alerts.
- **Pre-flight agent:** check a new mission and parameters before arming.
- **RAG over ArduPilot documentation:** for parameter-tuning advice with citations.
- **Human-in-the-loop approval:** approve or reject each proposed edit in the dashboard before the file is written.
- **Tool calling:** let a stronger model request extra analyses (for example a per-leg wind estimate) through strict tool schemas.
- **Better observability and deployment:** traces of every LLM call, and a hosted deployment with authentication.
- **Integration:** a Mission Planner plug-in button that sends the latest log to AeroMind.

CHAPTER 12 – CONCLUSION

This project addressed a practical problem: after a drone flight the answers to “how did it go, what went wrong, and what should I change” are buried in telemetry that few operators can read. AeroMind turns one flight's files into a verified one-page analysis and a corrected mission file.

The solution combines deterministic tools with a planned team of LLM agents. Python computes every metric, status, prediction and risk score. An orchestrator plans which of five specialist agents to run, and the agents explain, diagnose, assess and advise. A deterministic Verifier checks every claim and number against the data and triggers one self-correction round. A flight history gives the system long-term memory. Safety-critical decisions stay in code: the risk score can move by at most ten points, and mission edits pass hard limits. Nothing is uploaded to the drone. The system is built in Python with Pydantic, Streamlit, Plotly and Jinja2, and runs its agents on Claude or on a local Qwen2.5-3B model.

Evaluated on eight simulated flights with injected faults and known ground truth, AeroMind's agents detected every injected fault with the correct top cause, located both telemetry dropouts within 0.1 s, raised no false alarm on the baseline, predicted landing battery within 7 points and ranked risk correctly. The missions they corrected improved the next flight: the battery at landing of the undersized-battery flight rose from 14 % to 25 %, and the cross-track error of the wind flight fell from 1.8 m to 1.44 m.

The main lesson is that agents become trustworthy when their output is checked. Designing data contracts, guards and a deterministic Verifier first, and letting the agents add planning, explanation and advice on top, made the system both useful and reliable, even with a small model running locally. The project also showed the value of careful measurement: understanding the simulator, the sensors and the flight phases was as important as the agents themselves.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in Proc. International Conference on Learning Representations (ICLR), 2023.

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger and C. Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv:2308.08155, 2023.

[3] S. Hong et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in Proc. ICLR, 2024.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[5] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” in Advances in NeurIPS, 2023.

[6] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” ACM Computing Surveys, vol. 55, no. 12, 2023.

[7] A. Koubâa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith and M. Khalgui, “Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey,” IEEE Access, vol. 7, pp. 87658–87680, 2019.

[8] ArduPilot Dev Team, “ArduPilot Documentation: Copter, SITL, MAVProxy and Log Analysis,” ardupilot.org, 2026.

[9] ArduPilot Dev Team, “Mission Planner Documentation,” ardupilot.org/planner, 2026.

[10] ArduPilot, “UAV Log Viewer,” plot.ardupilot.org, 2026.

[11] L. Wang et al., “A Survey on Large Language Model based Autonomous Agents,” Frontiers of Computer Science, vol. 18, no. 6, 2024.

[12] PX4 Autopilot, “Flight Review and Log Analysis,” docs.px4.io, 2026.

[13] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.

[14] Anthropic, “Claude API Documentation: Messages and Structured Outputs,” docs.anthropic.com, 2026.

[15] Ollama, “Ollama Documentation and API,” ollama.com and github.com/ollama/ollama, 2026.

[16] ArduPilot, “pymavlink” and “MAVProxy,” github.com/ArduPilot, 2026.

[17] Ministry of Civil Aviation, Government of India, “The Drone Rules, 2021,” Gazette of India, 2021.

[18] Open Robotics, “Gazebo Harmonic Documentation,” gazebosim.org, 2026.

[19] Pydantic, “Pydantic Documentation (v2),” docs.pydantic.dev, 2026.

[20] Snowflake Inc., “Streamlit Documentation,” docs.streamlit.io, 2026.

[21] Plotly Technologies Inc., “Plotly Python Graphing Library,” plotly.com/python, 2026.

[22] The pandas development team, “pandas,” pandas.pydata.org, 2026; Pallets, “Jinja,” jinja.palletsprojects.com, 2026.

[23] OpenStreetMap contributors, “OpenStreetMap,” openstreetmap.org (map tiles in the Track tab), 2026.

[24] Simulation assets: Aero Sense disaster world and Hexa-X airframe (the team's Smart India Hackathon 2026 project); tdf_gazebo models (MIT licence); darpa_subt_worlds models (MIT licence); ardupilot_gazebo plugin and meshes (LGPL-3.0).

APPENDIX

Appendix A – Source Code

The complete source code, the demo flights and the cached analyses are in the project repository. The commands below reproduce every result in this report.

GitHub Repository: _____

Deployed Link: Not deployed (runs locally: `streamlit run app.py`)

```
pip install -e .                            # dependencies
python3 run.py --batch data/                # analyse all flights (fills
history.json, outputs/)
python3 run.py --compare outputs/s2 outputs/s2_fixed
python3 run.py --results                    # success metrics -> docs/RESULTS.md
streamlit run app.py                        # dashboard
python3 -m pytest -q                        # 236 tests
python3 -m sim.fly_scenario s0 s1 s2 s3 s4   # re-fly the scenarios (needs
Gazebo + SITL)
```

Appendix B – Additional Screenshots

Figures 10–12 show the Track, Telemetry and History tabs referred to in Section 7.6; Figures 13 and 14 show the Power & attitude tab for S2 and the Summary tab for S3.

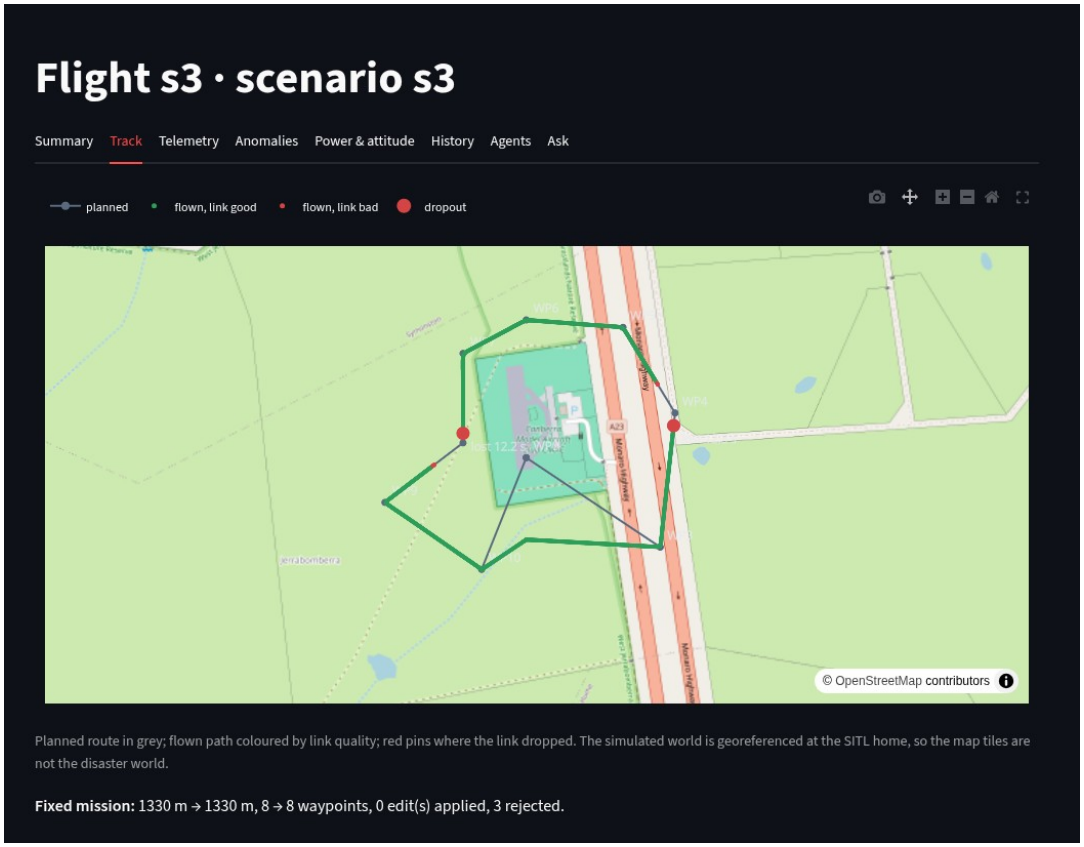


Figure 10: Track tab (S3): planned route, flown path coloured by link quality, dropout pins



Figure 11: Telemetry tab (S3): link strip with both dropouts, message rate, dropout table



Figure 12: History tab (S2 fixed): baseline, trends and what worked before

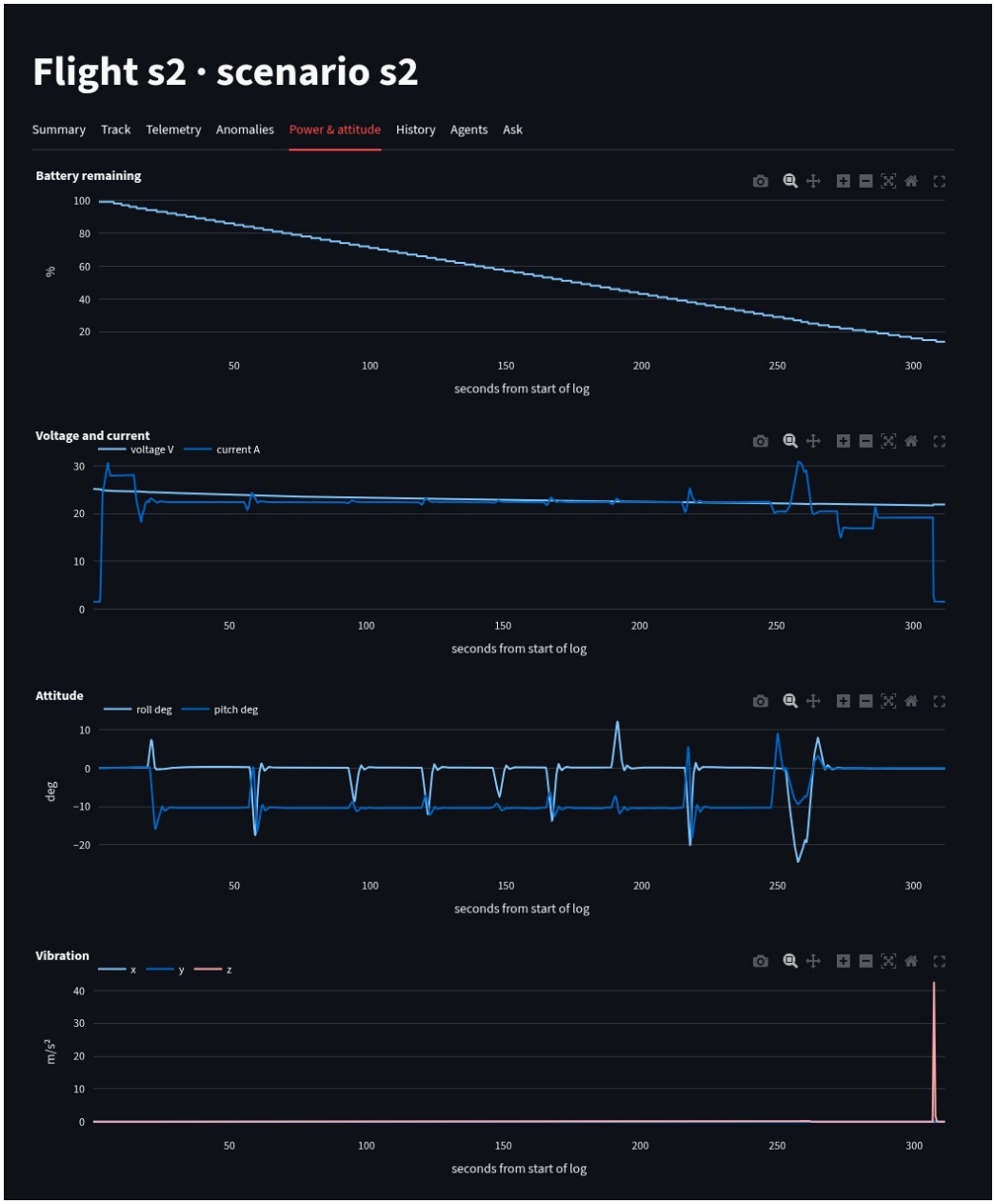


Figure 13: Power & attitude tab (S2): battery falling to 14 %

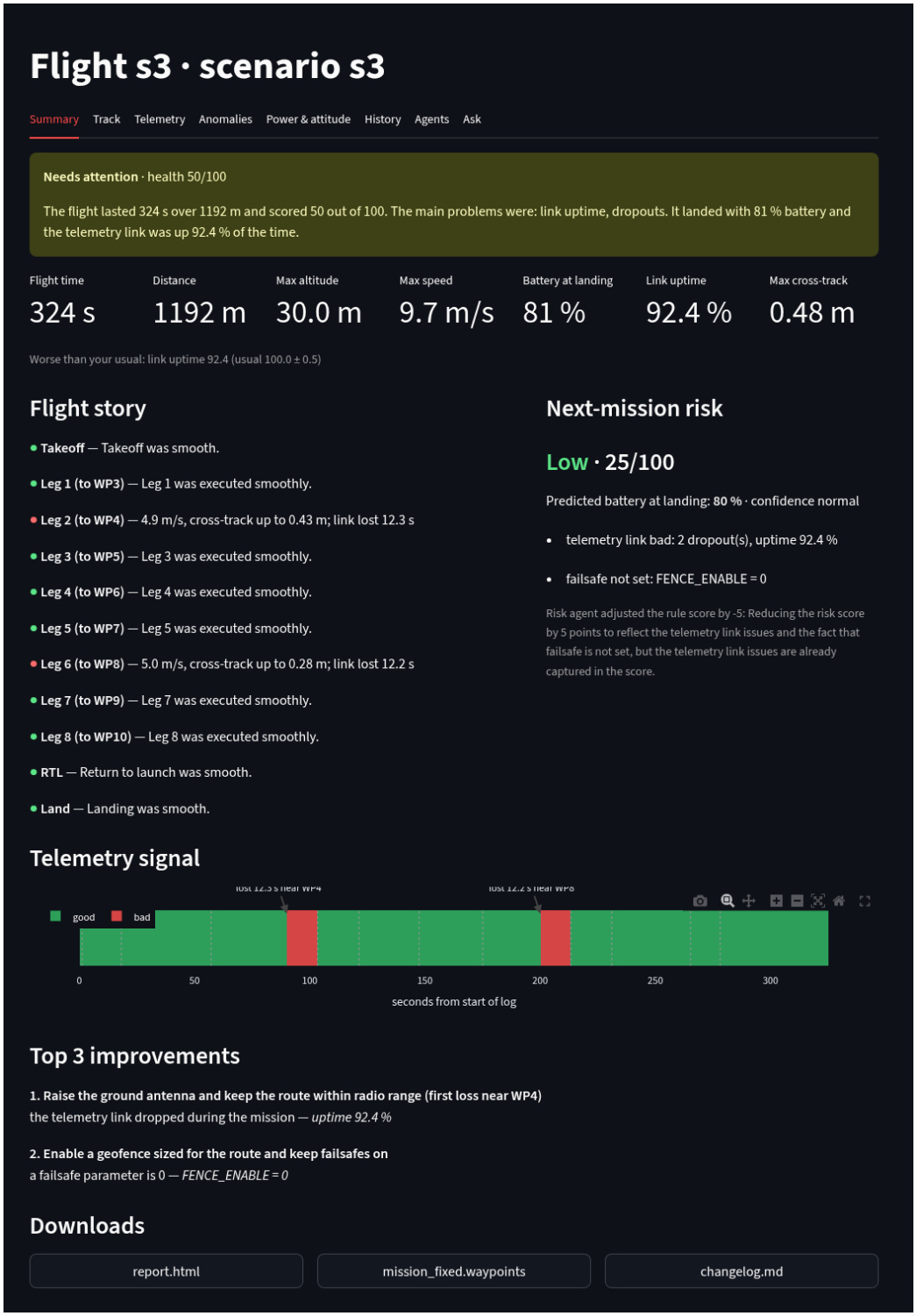


Figure 14: Summary tab (S3): both dropouts on the telemetry strip

Appendix C – Sample Prompts

Table 18 lists the prompts used by the system.

Table 18: Sample prompts (full text in agents/prompts.py and dashboard/ask.py)

Sr. No.	Prompt (abbreviated)	Purpose
1	You are a drone flight-test engineer writing for a beginner pilot. Give	Flight Summary

Sr. No.	Prompt (abbreviated)	Purpose
	a verdict, one short line per flight phase and a 3-sentence summary… Use only the JSON you are given. Copy numbers exactly…	agent
2	You are a telemetry radio engineer. Describe link quality across the whole flight. For each dropout give its time, duration and nearest waypoint as a finding citing link.dropouts[<index>].duration_s…	Telemetry Link agent
3	You are a drone failure analyst. List findings that cite metric names and values… up to 3 root causes… Start from candidate_causes… Use pilot notes as supporting evidence only.	Diagnosis agent
4	You are a mission risk assessor… You may adjust it by at most 10 points… and must explain why…	Risk agent
5	You are a mission planner. Propose mission edits from the allowed actions only, each tied to a cause id… A link_loss cause gets advice, never an edit…	Advisor agent
6	You are the orchestrator of a post-flight analysis. From the manifest, choose which agents run and in what order, with a one-line reason each…	Orchestrator planner
7	You answer a pilot's question about one drone flight using only the JSON given. Cite the metric paths you used…	Ask tab

Appendix D – Test Data

Table 19 lists the flights used for testing; all are in the repository under `data/`.

Table 19: Test data: flights in data/ (each with mission.waypoints, params.param, truth.json, meta.json)

Folder	Scenario	tlog files	Flight s	Parent	Injected outages
s0	Baseline	1	347	–	0
s1	Strong wind + fast flight	1	274	–	0
s2	Undersized battery	1	310	–	0
s3	Telemetry dropout	3	324	–	2
s4	Combined (wind + fast + small battery + radio shadow)	1	231	–	1
s1_fixed	Strong wind + fast flight	1	282	s1	0
s2_fixed	Undersized battery	1	273	s2	0
s4_fixed	Combined (wind + fast + small battery + radio shadow)	2	280	s4	1